Understanding Two's Complement

Rizvan Ali Follow 3 min read · Aug 18, 2023

In the world of digital computers, data representation is crucial. One of the most fundamental techniques used to represent signed integers is the two's complement method. In this article, we'll dive deep into the concept of two's complement, explore its implementation in Java, and understand the implications of integer wraparound.

Two's Complement: The Basics

Two's complement is a method used to represent negative integers in binary form. In digital computers, binary representation is used because it's based on two states, which can be represented by high and low voltage levels. While positive integers can be straightforwardly represented in binary, representing negative integers requires a scheme like two's complement.

Here's a step-by-step process to find the two's complement of a binary number:

1. **Write down the binary number:** Start with the binary representation of the positive integer you wish to negate.
2. **Invert all the bits:** Change all the 1s to 0s and all the 0s to 1s.
3. **Add 1 to the result:** Add 1 to the binary number obtained from the previous step.

The beauty of two's complement is that it allows for simple binary arithmetic. When you add a number to its two's complement, you'll get a result of all zeroes. Moreover, in systems that use two's complement, subtracting numbers can be done by simply adding the two's complement of the number to be subtracted.

Positive Numbers

Positive numbers in two's complement are represented just like regular binary numbers. The most significant bit (MSB) is 0, indicating that the number is positive.

For instance, the number 55 in binary is 01010101 for a 4-bit representation.

Negative Numbers

Negative numbers are where the two's complement technique comes into play. To represent a negative number:

1. Convert the magnitude of the number to binary.
2. Invert all the bits.
3. Add 1 to the result.

2. Invert all bits: 10101010

3. Add 1: 1010+1=10111010+1=1011

So, −5−5 is represented as 10111011 in a 4-bit two's complement system.

Example 2: -7 in 4-bit two's complement

1. Binary of 7 (magnitude): 01110111

2. Invert all bits: 10001000

3. Add 1: 1000+1=10011000+1=1001

Thus, −7−7 is represented as 10011001 in a 4-bit two's complement system.

Arithmetic in Two's Complement

The beauty of two's complement is that it makes arithmetic operations straightforward.

For instance, to subtract B from A (i.e. $A−B$), you can:

1. Convert B to its two's complement (which represents $−B$).

2. Add this value to A.

3. If there's a carry out of the most significant bit, discard it.

Example: Subtract 5 from 3 using 4-bit two's complement

1. Represent 3 in binary: 00110011
2. Find two's complement of 5 (from the previous example): 10111011
3. Add the two numbers:

$$\begin{array}{rl} & 0011 \text{ (This is 3)} \\ + & 1011 \text{ (This is -5)} \\ \hline & 1110 \text{ (This represents -2)} \end{array}$$

The result is 1110, which is the 4-bit two's complement representation of -2. So, $3 - 5 = -2$ as expected!

Advantages of Two's Complement:

1. Unified Addition/Subtraction: As demonstrated, addition and subtraction can be performed using the same hardware, simplifying digital circuit design.
2. Unique Representations: Every value has a unique representation. There's no positive and negative zero as in one's complement; there's only one zero.

Noteworthy Points:

1. MSB as the Sign Bit: In two's complement representation, the most significant bit (MSB) acts as the sign bit. If the MSB is 0, the number is positive. If the MSB is 1, the number is negative.

2. Range: An n-bit two's complement number can represent integers from

$$-2^{n-1} \leq x \leq 2^{n-1} - 1$$

For example, an 8-bit two's complement number can represent integers from -128 to 127.